

Lab 8

Set up a new channel named as Ultrasonic sensor. Add a new field named as distance to your channel. To visualize the data on cloud, add widgets of your choice. Interface ultrasonic sensor with NodeMCU and upload data received on serial monitor on ThingSpeak.

```
#include <ESP8266WiFi.h>          // Library to connect NodeMCU with WiFi
#include <ESP8266HTTPClient.h>    // Library to send HTTP requests

const char* ssid = "YOUR_WIFI_NAME"; // WiFi name
const char* password = "YOUR_WIFI_PASSWORD"; // WiFi password

String writeAPIKey = "YOUR_WRITE_API_KEY"; // ThingSpeak Write API key
String server = "http://api.thingspeak.com/update"; // ThingSpeak update URL

WiFiClient client;

#define TRIG D5 // Ultrasonic TRIG pin
#define ECHO D6 // Ultrasonic ECHO pin

void setup() {

    Serial.begin(115200); // Start serial monitor

    pinMode(TRIG, OUTPUT); // Set TRIG pin as output
    pinMode(ECHO, INPUT); // Set ECHO pin as input

    WiFi.begin(ssid, password); // Connect to WiFi

    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("WiFi Connected");
}

void loop() {

    // Trigger ultrasonic sensor
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    // Measure echo duration
    long duration = pulseIn(ECHO, HIGH);

    // Convert to distance (cm)
    float distance = duration * 0.034 / 2;

    Serial.print("Distance: ");
    Serial.println(distance);

    // Send data to ThingSpeak
```

```

if (WiFi.status() == WL_CONNECTED) {

    HTTPClient http;

    // Create URL with API key and distance value
    String url = server + "?api_key=" + writeAPIKey + "&field1=" + String(distance);

    http.begin(client, url);    // Start HTTP connection

    int httpCode = http.GET(); // Send GET request

    if (httpCode > 0) {
        Serial.print("HTTP Response Code: ");
        Serial.println(httpCode);
    }
    else {
        Serial.print("Error sending message: ");
        Serial.println(http.errorToString(httpCode));
    }

    http.end(); // Close connection
}

delay(15000); // ThingSpeak requires 15 sec delay
}

```

Interface a ultrasonic sensor with NodeMCU. Set the threshold value for your distance as 25 cm.

Upload the sensor data on ThingSpeak and set the indicator color green for distance below threshold value and red for the values above threshold.

```

#include <ESP8266WiFi.h>
#include <ThingSpeak.h>

const char* ssid = "YOUR_WIFI";
const char* password = "YOUR_PASSWORD";

WiFiClient client;

unsigned long channelID = YOUR_CHANNEL_ID;
const char* writeAPIKey = "YOUR_WRITE_API_KEY";

#define TRIG D5
#define ECHO D6

int threshold = 25;    // threshold distance in cm

void setup() {

    Serial.begin(115200);

    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);

    WiFi.begin(ssid, password);

    Serial.print("Connecting");

```

```

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("Wi-Fi Connected");

ThingSpeak.begin(client);
}

void loop() {

    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    long duration = pulseIn(ECHO, HIGH);

    float distance = duration * 0.034 / 2;

    Serial.print("Distance: ");
    Serial.println(distance);

    int statusValue;

    if(distance < threshold)
    {
        statusValue = 0;    // Safe
        Serial.println("Below Threshold");
    }
    else
    {
        statusValue = 1;    // Alert
        Serial.println("Above Threshold");
    }

    ThingSpeak.setField(1, distance);
    ThingSpeak.setField(2, statusValue);

    int x = ThingSpeak.writeFields(channelID, writeAPIKey);

    if(x == 200)
    {
        Serial.println("Data Uploaded");
    }
    else
    {
        Serial.println("Upload Failed");
    }

    delay(15000);
}

```

2nd way

```
include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

const char* ssid = "YOUR_WIFI"; // Your WiFi SSID
const char* password = "YOUR_PASSWORD"; // Your WiFi password

String writeAPIKey = "YOUR_WRITE_API_KEY"; // ThingSpeak Write API key
String server = "http://api.thingspeak.com/update"; // ThingSpeak server URL

WiFiClient client;

#define TRIG D5
#define ECHO D6

int threshold = 25; // Distance threshold in cm

void setup() {
    Serial.begin(115200);

    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);

    WiFi.begin(ssid, password);
    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("WiFi Connected");
}

void loop() {

    // Trigger the ultrasonic sensor
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    // Read the echo time
    long duration = pulseIn(ECHO, HIGH);
    float distance = duration * 0.034 / 2; // Convert to cm

    Serial.print("Distance: ");
    Serial.println(distance);

    // Determine status using regular if-else
    int statusValue;
    if (distance < threshold) {
        statusValue = 0; // Green = safe
        Serial.println("Below Threshold (Green)");
    } else {
        statusValue = 1; // Red = alert
        Serial.println("Above Threshold (Red)");
    }
}
```

```
// Send data to ThingSpeak using HTTP GET
if (WiFi.status() == WL_CONNECTED) {

    HTTPClient http;

    String url = server + "?api_key=" + writeAPIKey + "&field1=" + String(distance) + "&field2="
    + String(statusValue);

    http.begin(client, url);
    int httpCode = http.GET();

    if (httpCode > 0) {
        Serial.print("HTTP Response Code: ");
        Serial.println(httpCode);
    } else {
        Serial.print("Error sending message: ");
        Serial.println(http.errorToString(httpCode));
    }

    http.end(); // Close HTTP connection
}

delay(15000); // ThingSpeak requires 15 sec delay
}
```